CompTIA Certified Linux+ Training (XK0-006)

WSQ Training · CompTIA Linux+ XK0-006 V8

Course Code: TGS-2024048316

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Trainer: Dr. Alfred Ang | Version 1.0

Administration

Digital Attendance

- Take the AM, PM and Assessment digital attendance on every training day.
- Scan the attendance QR code on the LMS (TRAQOM) for each session.
- Attendance of at least 75% is required for funding and certification.
- LMS: https://lms-tms.tertiaryinfotech.com/

About the Trainer

General Trainer (template)

Name

Title

Qualifications

Areas of Expertise

Industry Experience

Contact

This blank template can be completed by any assigned trainer.

About the Trainer

Your Trainer

Name	Dr. Alfred Ang
Title	Principal Trainer & Consultant, Tertiary Infotech Academy
Qualifications	PhD; ACTA/ACLP-certified adult educator
Areas of Expertise	Linux systems administration, cloud, DevOps, cybersecurity, AI
Industry Experience	20+ years in IT training, software and infrastructure engineering
Contact	enquiry@tertiaryinfotech.com · +65 6100 0613

Let's Know Each Other

Learner Introduction

- Your name and organisation / role.
- Your current experience with Linux (beginner → advanced).
- Which of the five domains matters most to your day job.
- What you want to walk away able to do — and any exam timeline.

Ground Rules

- Be punctual for each session and after every break.
- Set devices to silent; give the hands-on labs your full attention.
- Ask questions any time — the labs are best learned by doing.
- Do the labs yourself on Killercoda; typing builds exam-day muscle memory.
- Respect fellow learners and keep shared environments clean (reset between labs).

Course Outline

Five Exam Domains · 30 Hands-on Labs

23%

Domain 1 — System Management

Labs 1–7 · objectives 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7

20%

Domain 2 — Services and User Management

Labs 8–13 · objectives 2.1, 2.2, 2.3, 2.4, 2.5, 2.6

18%

Domain 3 — Security

Labs 14–19 · objectives 3.1, 3.2, 3.3, 3.4, 3.5, 3.6

17%

Domain 4 — Automation, Orchestration, and Scripting

Labs 20–24 · objectives 4.1, 4.2, 4.3, 4.4, 4.5

22%

Domain 5 — Troubleshooting

Labs 25–30 · objectives 5.1, 5.2, 5.3, 5.4, 5.5

Administration

Lesson Plan — 2 Days (9:00 AM – 6:00 PM)

Day 1	Domain 1 — System Management (23%) · Domain 2 — Services & User Management (20%) | Labs 1–13
Day 2 AM	Domain 3 — Security (18%) · Domain 4 — Automation, Orchestration & Scripting (17%) | Labs 14–24
Day 2 PM	Domain 5 — Troubleshooting (22%) · Capstone | Labs 25–30
Day 2 · 4:00–6:00 PM	Final Assessment — Written (SAQ) 1 hr + Practical Performance (PP) 1 hr (Open Book)

Each day = 8 instructional hours (1-hour lunch; tea breaks within). Breaks may compress to make room for the assessment block.

Briefing for Assessment

- The assessment is on Day 2, 4:00 PM – 6:00 PM, and is OPEN BOOK.
- Written Assessment (SAQ) — 1 hour (4:00–5:00 PM): short-answer knowledge questions.
- Practical Performance (PP) — 1 hour (5:00–6:00 PM): hands-on lab tasks.
- Open book = you may use the slides, Learner Guide and approved materials.
- Flow: TRAQOM survey → Assessment Digital Attendance → Assessment → Submit on LMS → Sign the Assessment Summary Record.
- If assessed Not Yet Competent, an appeal and re-assessment process is available.

Assessment & Funding

- Be assessed Competent (C) in BOTH the Written Assessment and the Practical Performance.
- Attend at least 75% of the course.
- Complete the TRAQOM survey and sign the Assessment Summary Record.
- Courseware download and assessment submission are on the LMS: https://lms-tms.tertiaryinfotech.com/

Learning Outcomes

1. Explain core Linux concepts — the boot process, the Filesystem Hierarchy Standard, device management and virtualization — and manage storage, networking and shell operations.

2. Manage files, local user and group accounts, processes and jobs, software packages, systemd services and containerized applications on a Linux server.

3. Apply Linux security: authentication/authorization (sudo, PAM), firewalls, OS and account hardening, cryptography and compliance/audit procedures.

4. Automate administration with Ansible, Bash and Python, apply Git version control, and use AI assistants responsibly and securely.

5. Monitor a Linux system and analyze and troubleshoot hardware, storage, network, security and performance issues using the right diagnostic tools.

DOMAIN 1

System Management

23% of the exam · Labs 1–7

1.1 Explain basic Linux concepts

1.2 Summarize Linux device management concepts and tools

1.3 Given a scenario, manage storage in a Linux system

1.4 Given a scenario, manage network services and configurations

1.5 Given a scenario, manage a Linux system using common shell operations

1.6 Given a scenario, perform backup and restore operations

1.7 Summarize virtualization on Linux systems

Boot Process & Filesystem Hierarchy

Maps to CompTIA Linux+ objective 1.1 — Explain basic Linux concepts

Goal

Inspect a running Linux system's boot configuration, kernel command line, initramfs, and the standard Filesystem Hierarchy to understand how GRUB hands off to the kernel and where FHS directories live.

What you'll do

Identify the distro and kernel, inspect GRUB, the kernel cmdline, and initramfs, then tour the FHS and boot-related directories.

KEY COMMANDS & CONCEPTS

- `/etc/os-release`
- `GRUB2`
- `/proc/cmdline`
- `initramfs`
- `lsinitramfs`
- `FHS`
- `systemd targets`
- `usrmerge`

✓ **Test it:** You can identify the running distro/kernel/arch and explain how GRUB, the kernel cmdline, and initramfs bring up userspace.

Lab 1 · 1.1

Boot Process & Filesystem Hierarchy — Commands

Identify distribution and architecture

```
cat /etc/os-release
uname -a
uname -m
uname -r
hostnamectl 2>/dev/null || true
lsb_release -a
```

Inspect GRUB configuration

```
cat /etc/default/grub 2>/dev/null || true
ls /boot 2>/dev/null
find / -name "grub.cfg" 2>/dev/null | head
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 2 · Objective 1.2

Kernel Modules & Device Management

Maps to CompTIA Linux+ objective 1.2 — Summarize Linux device management concepts and tools

Goal

List, inspect, load, and unload kernel modules and use the standard hardware-enumeration tools to understand modprobe vs insmod, module dependencies, and reading dmesg.

What you'll do

Install discovery tools, inspect and load/unload a module, examine module dependencies, read the kernel ring buffer, and enumerate hardware.

KEY COMMANDS & CONCEPTS

- `lsmod`
- `modinfo`
- `modprobe`
- `insmod/rmmod`
- `depmod`
- `dmesg`
- `lspci/lsusb/lscpu/lsblk`
- `dracut vs mkinitramfs`

✓ **Test it:** You can pick the right ls* tool for a hardware question and explain insmod vs modprobe vs depmod.

Lab 2 · 1.2

Kernel Modules & Device Management — Commands

Install hardware inspection tools

```
apt-get update -qq
apt-get install -y pciutils usbutils hwinfo lshw dmidecode kmod >/dev/null
```

List loaded modules and inspect one

```
lsmod | head -20
lsmod | wc -l
MOD=$(lsmod | awk 'NR==2 {print $1}')
modinfo "$MOD" | head -20
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 3 · Objective 1.3

Storage with LVM, Partitions & Filesystems

Maps to CompTIA Linux+ objective 1.3 — Manage storage in a Linux system

Goal

Build a complete LVM stack on loopback-backed disks, format with ext4/xfs/btrfs, mount with hardening options, persist in fstab, and grow a filesystem online.

What you'll do

Create loopback disks, partition, build PVs/VG/LVs, make filesystems, mount with hardening options into fstab, then extend an LV and its filesystem live.

KEY COMMANDS & CONCEPTS

- `losetup`
- `parted/GPT`
- `pvcreate/vgcreate/lvcreate`
- `mkfs.ext4/xfs/btrfs`
- `blkid/UUID`
- `/etc/fstab`
- `mount hardening (nodev/nosuid/noexec)`
- `lvextend/resize2fs/xfs_growfs`

✓ **Test it:** You can go from raw block device to a mounted, UUID-referenced, online-expandable filesystem; note XFS grows but cannot shrink.

Lab 3 · 1.3

Storage with LVM, Partitions & Filesystems — Commands

Install tooling and create loopback disks

```
apt-get install -y lvm2 xfsprogs btrfs-progs parted util-linux >/dev/null
mkdir -p /lab/disks && cd /lab/disks
for i in 1 2 3; do
  truncate -s 512M disk${i}.img
  losetup -fP disk${i}.img
done
losetup -a
```

Partition the first loopback disk

```
LOOP1=$(losetup -j /lab/disks/disk1.img | cut -d: -f1)
parted -s "$LOOP1" mklabel gpt
parted -s "$LOOP1" mkpart primary 1MiB 100%
parted -s "$LOOP1" set 1 lvm on
partprobe "$LOOP1"
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 4 · Objective 1.4

Network Configuration

Maps to CompTIA Linux+ objective 1.4 — Manage network services and configurations on a Linux server

Goal

Inspect interfaces, addresses, and routes with ip; walk the name-resolution stack; validate a netplan YAML; and use the standard diagnostic toolbox to read a Linux network stack top-to-bottom.

What you'll do

Install network tools, inspect interfaces/routes, walk the resolution stack, run DNS diagnostics, validate netplan, check sockets/connectivity, and capture packets/scan ports.

KEY COMMANDS & CONCEPTS

- `ip -br link/address/route`
- `/etc/nsswitch.conf`
- `/etc/resolv.conf`
- `getent`
- `dig/nslookup`
- `netplan generate`
- `ss`
- `tcpdump/nmap`

✓ **Test it:** You can read the resolution path (nsswitch -> hosts/resolv.conf -> DNS) and pick the right diagnostic: ss, dig/getent, tcpdump, nmap.

Lab 4 · 1.4

Network Configuration — Commands

Install tools

```
apt-get update -qq
apt-get install -y iproute2 iputils-ping dnsutils curl mtr-tiny tcpdump traceroute nmap network-manager netplan.io
>/dev/null
```

Interfaces, addresses, routes

```
ip -br link
ip -br address
ip route
ip -6 route
cat /proc/net/dev | head
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 5 · Objective 1.5

Shell Operations & Text Processing

Maps to CompTIA Linux+ objective 1.5 — Manage a Linux system using common shell operations

Goal

Work with shell environment variables, login vs interactive startup files, every common redirection form, and the classic text-processing pipeline to assemble non-trivial one-liners.

What you'll do

Explore env vars and PATH, paths and startup files, all redirection operators, then grep/cut/sort/uniq, sed/awk, tr/xargs, and scripted editors.

KEY COMMANDS & CONCEPTS

- `export/env/unset`
- `login vs interactive shells`
- `redirection > >> < << <<< | tee`
- `grep/cut/sort/uniq/wc`
- `sed/awk`
- `tr/xargs`
- `shell scripting`

✓ **Test it:** You can compose grep/awk/sed/cut/sort/uniq/tr/xargs into ad-hoc pipelines and know which startup file each shell type sources.

Lab 5 · 1.5

Shell Operations & Text Processing — Commands

Environment variables and PATH

```
echo "USER=$USER  HOME=$HOME  SHELL=$SHELL"
echo "PATH=$PATH"
env | head
export LABVAR="hello linux+"
env | grep LABVAR
unset LABVAR
```

Absolute vs relative paths and startup files

```
mkdir -p /tmp/lab5 && cd /tmp/lab5
pwd
ls ./
ls ../tmp
cat ~/.bashrc 2>/dev/null | head || echo "no .bashrc"
cat /etc/profile | head
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 6 · Objective 1.6

Backup & Restore

Maps to CompTIA Linux+ objective 1.6 — Perform backup and restore operations for a Linux server

Goal

Create tar/cpio/dd archives with gzip/bzip2/xz compression, image and recover disks with dd and ddrescue, mirror trees with rsync, and verify integrity with sha256sum.

What you'll do

Make sample data, archive with tar and cpio, dd a disk image, recover a damaged one with ddrescue, mirror with rsync --delete, then verify checksums and read compressed files in place.

KEY COMMANDS & CONCEPTS

- `tar (czf/xzf/tzf)`
- `cpio`
- `dd`
- `ddrescue`
- `rsync --delete`
- `sha256sum -c`
- `zcat/zgrep/gzip -l`
- `gzip/bzip2/xz tradeoffs`

✓ **Test it:** You can choose tar/cpio/dd/ddrescue/rsync appropriately and know sha256sum -c and rsync trailing-slash/--delete semantics.

Backup & Restore — Commands

Install tooling and create sample data

```
apt-get install -y tar cpio gzip bzip2 xz-utils rsync gddrescue coreutils >/dev/null
mkdir -p /lab6/src/{etc,logs,docs}
cp -r /etc/hostname /etc/os-release /lab6/src/etc/
for i in 1 2 3 4 5; do
  dd if=/dev/urandom of=/lab6/src/logs/log${i}.bin bs=1K count=20 status=none
done
echo "important document" > /lab6/src/docs/notes.txt
```

tar with gzip, bzip2, and xz

```
cd /lab6
tar -czvf src.tar.gz  src/ 2>&1 | tail -5
tar -cjvf src.tar.bz2 src/ 2>&1 | tail -2
tar -cJvf src.tar.xz  src/ 2>&1 | tail -2
tar -tzf src.tar.gz | head -5
tar -xzvf src.tar.gz -C restore/ 2>&1 | tail -3
diff -r src restore/src && echo "restore matches source"
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 7 · Objective 1.7

Virtualization with QEMU/KVM & libvirt

Maps to CompTIA Linux+ objective 1.7 — Summarize virtualization on Linux systems

Goal

Install QEMU and libvirt, manipulate virtual disk images with qemu-img, list libvirt domains/networks with virsh, and boot a tiny guest under QEMU to discuss hypervisors, image formats, snapshots, and network modes.

What you'll do

Install QEMU/libvirt, create and inspect disk images, convert/resize them, take qcow2 snapshots, list libvirt domains/networks, boot a guest under TCG, and review the three network modes.

KEY COMMANDS & CONCEPTS

- `QEMU vs KVM`
- `/dev/kvm vs TCG`
- `qemu-img (qcow2/raw)`
- `qemu-img convert/resize`
- `qcow2 snapshots`
- `virsh list/net-list`
- `bridged/NAT/host-only networks`

✓ **Test it:** You can distinguish QEMU (emulator) from KVM (accelerator), qcow2 from raw, internal vs libvirt snapshots, and the three guest network modes.

Lab 7 · 1.7

Virtualization with QEMU/KVM & libvirt — Commands

Install QEMU and libvirt

```
apt-get install -y qemu-system-x86 qemu-utils libvirt-clients libvirt-daemon-system bridge-utils >/dev/null
ls -l /dev/kvm 2>/dev/null && echo "KVM available" || echo "KVM NOT available - use TCG"
qemu-system-x86_64 --version | head -1
virsh --version
```

Create and inspect disk images

```
mkdir -p /lab7 && cd /lab7
qemu-img create -f qcow2 disk.qcow2 1G
qemu-img create -f raw   disk.raw   256M
qemu-img info disk.qcow2
qemu-img info disk.raw
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

DOMAIN 2

Services and User Management

20% of the exam · Labs 8–13

2.1 Given a scenario, manage files and directories

2.2 Given a scenario, perform local account management

2.3 Given a scenario, manage processes and jobs

2.4 Given a scenario, configure and manage software

2.5 Given a scenario, manage Linux using systemd

2.6 Given a scenario, manage applications in a container

Lab 8 · Objective 2.1

File & Directory Management

Maps to CompTIA Linux+ objective 2.1 — Manage files and directories on a Linux system

Goal

Practice core file and directory operations: navigating, creating, copying, searching, linking, and inspecting special device files with standard Linux utilities.

What you'll do

Create a lab tree, then copy, find, link, diff, and inspect files and /dev device nodes.

KEY COMMANDS & CONCEPTS

- `ls -la / stat / file`
- `find predicates (-name, -mtime, -size)`
- `locate & updatedb`
- `lsof`
- `diff & sdiff`
- `hard vs symbolic links`
- `block/character device files`

✓ **Test it:** ls -li confirms hardlink shares the target's inode while the symlink breaks after the target is removed.

Lab 8 · 2.1

File & Directory Management — Commands

Navigate and list

```
pwd
cd /tmp
mkdir -p lab08/{src,dst,archive}
cd lab08
ls -la
```

Create, copy, move, touch

```
touch src/file1.txt src/file2.log
echo "hello" > src/file1.txt
cp -a src/file1.txt dst/
mv src/file2.log archive/
stat dst/file1.txt
file dst/file1.txt
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 9 · Objective 2.2

Local Account & Group Management

Maps to CompTIA Linux+ objective 2.2 — Perform local account management in a Linux environment

Goal

Create and manage local users and groups, set passwords and aging policies, change shells, and distinguish user, system, and service accounts.

What you'll do

Create users/groups, set passwords and aging, inspect identity databases, then remove the accounts.

KEY COMMANDS & CONCEPTS

- `/etc/passwd, /etc/shadow, /etc/group, /etc/skel`
- `useradd / adduser / usermod -aG`
- `chpasswd, passwd -S, chage`
- `chsh, getent, id`
- `UID>=1000 user vs UID<1000 system vs service (-r)`
- `userdel -r / groupdel`

✓ **Test it:** getent passwd bob returns nothing after userdel -r, confirming the account and home directory are gone.

Lab 9 · 2.2

Local Account & Group Management — Commands

Inspect account databases

```
head -5 /etc/passwd
head -5 /etc/shadow
head -5 /etc/group
ls /etc/skel -la
head -20 /etc/profile
```

Create users and groups

```
groupadd developers
useradd -m -s /bin/bash -G developers alice
adduser --disabled-password --gecos "" bob
usermod -aG developers bob
id alice
groups bob
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 10 · Objective 2.3

Processes, Jobs & Scheduling

Maps to CompTIA Linux+ objective 2.3 — Manage processes and jobs in a Linux environment

Goal

Inspect processes, manipulate shell jobs, adjust priorities, send signals, and schedule tasks with at, cron, and anacron.

What you'll do

Examine processes via ps/proc/strace, control background jobs, renice, signal, and schedule jobs.

KEY COMMANDS & CONCEPTS

- `ps auxf / pstree / pidstat / mpstat`
- `/proc/<PID> and lsof -p`
- `process states (R,S,D,Z,T) & strace`
- `jobs: &, Ctrl+Z, bg, fg, nohup, disown`
- `nice / renice priority`
- `kill / pkill / killall signals`
- `at, cron, anacron`

✓ **Test it:** atq lists the queued at job and crontab -l shows the recurring entry, confirming both schedulers accepted the tasks.

Lab 10 · 2.3

Processes, Jobs & Scheduling — Commands

Inspect running processes

```
ps auxf | head -20
pstree -p | head -20
apt update -y && apt install -y htop sysstat lsof strace at
pidstat 1 3
mpstat 1 2
```

Explore /proc and lsof

```
sleep 600 &
SPID=$!
ls /proc/$SPID/
cat /proc/$SPID/status | head
cat /proc/$SPID/cmdline; echo
lsof -p $SPID | head
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 11 · Objective 2.4

Software & Package Management

Maps to CompTIA Linux+ objective 2.4 — Configure and manage software in a Linux environment

Goal

Manage Debian packages with apt/dpkg, add a signed third-party repo, run dnf/rpm in a container, and use language and sandboxed package managers.

What you'll do

Install/remove apt packages, add a GPG-signed repo, demo dnf/rpm, then pip/npm/cargo and snap/flatpak.

KEY COMMANDS & CONCEPTS

- `apt update/install/remove/purge/autoremove`
- `apt-cache policy, dpkg -l / -L`
- `signed-by GPG keyrings`
- `dnf / rpm -q / rpm -V`
- `pip, npm -g, cargo`
- `update-alternatives`
- `snap & flatpak sandboxes`

✓ **Test it:** apt-cache policy terraform shows a candidate from the signed HashiCorp repo, confirming the GPG-verified source is trusted.

Lab 11 · 2.4

Software & Package Management — Commands

apt basics

```
apt update
apt install -y tree jq
apt-cache policy jq
dpkg -l | grep -E "tree|jq"
dpkg -L tree | head
```

Add a third-party repo with GPG key

```
apt install -y curl gnupg lsb-release
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://apt.releases.hashicorp.com/gpg | gpg --dearmor -o /etc/apt/keyrings/hashicorp.gpg
echo "deb [signed-by=/etc/apt/keyrings/hashicorp.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" >
/etc/apt/sources.list.d/hashicorp.list
apt update
apt-cache policy terraform | head
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 12 · Objective 2.5

Service Management with systemd

Maps to CompTIA Linux+ objective 2.5 — Manage Linux using systemd

Goal

Inspect and control systemd units, author custom service/timer/mount units, analyze boot performance, and configure host/time/DNS via systemd helpers.

What you'll do

Manage nginx, write a service + timer + mount unit, analyze boot time, and set hostname/timezone/DNS.

KEY COMMANDS & CONCEPTS

- `systemctl list-units / list-unit-files / --failed`
- `start/stop/restart/reload, enable/disable, mask/unmask`
- `custom .service (Type=oneshot) + daemon-reload`
- `.timer units (OnUnitActiveSec)`
- `.mount units`
- `systemd-analyze blame / critical-chain`
- `hostnamectl / timedatectl / resolvectl`
- `journalctl -u`

✓ **Test it:** systemctl list-timers shows lab-hello.timer scheduled and journalctl -u lab-hello records each firing, confirming the custom units work.

Lab 12 · 2.5

Service Management with systemd — Commands

List and inspect units

```
systemctl list-units --type=service | head -20
systemctl list-unit-files --type=service | head -20
systemctl --failed
```

Control nginx

```
apt update -y && apt install -y nginx
systemctl status nginx --no-pager
systemctl stop nginx; systemctl start nginx; systemctl restart nginx; systemctl reload nginx
systemctl disable nginx; systemctl enable nginx
systemctl mask nginx && systemctl start nginx 2>&1 | head
systemctl unmask nginx
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 13 · Objective 2.6

Containers with Docker & Podman

Maps to CompTIA Linux+ objective 2.6 — Manage applications in a container on a Linux server

Goal

Install Docker and Podman, pull/run images, manage volumes and networks, build an image from a Dockerfile, and contrast rootful vs rootless engines.

What you'll do

Run and inspect nginx containers, bind-mount volumes, build an image, wire a user-defined network, and try rootless Podman.

KEY COMMANDS & CONCEPTS

- `docker.io vs podman (daemonless/rootless)`
- `docker pull/run -d -p --name, docker ps`
- `logs / exec -it / inspect / stats`
- `bind-mount volumes (-v host:container:ro)`
- `Dockerfile: FROM/RUN/COPY/USER/ENTRYPOINT/CMD`
- `docker network create (built-in DNS)`
- `rootless Podman & --privileged risks`

✓ **Test it:** curl against the published ports returns 200 and the alpine container reaches http://api by name, confirming containers, volumes, and the user-defined network work.

Lab 13 · 2.6

Containers with Docker & Podman — Commands

Install Docker and Podman

```
apt update -y
apt install -y docker.io podman
systemctl start docker
docker --version
podman --version
```

Pull and run a container

```
docker pull nginx
docker images
docker run -d -p 8080:80 --name web nginx
docker ps
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

DOMAIN 3

Security

18% of the exam · Labs 14–19

3.1 Summarize authorization, authentication, and accounting methods

3.2 Given a scenario, configure and implement firewalls

3.3 Given a scenario, apply OS hardening techniques

3.4 Explain account hardening techniques and best practices

3.5 Explain cryptographic concepts and technologies

3.6 Explain the importance of compliance and audit procedures

Lab 14 · Objective 3.1

AAA: sudo, PAM, and Polkit

Maps to CompTIA Linux+ objective 3.1 — Summarize authorization, authentication, and accounting methods

Goal

Configure sudo authorization, inspect PAM authentication stacks, and enable auditd accounting to record security events. Finish with a conceptual look at centralized identity via LDAP, Kerberos, and SSSD.

What you'll do

Install sudo, grant a user privileges, inspect PAM, read auth logs, and enable auditd rules.

KEY COMMANDS & CONCEPTS

- `sudo / visudo / sudoers.d`
- `PAM stacks (auth, account, session)`
- `auditctl / ausearch`
- `journalctl / rsyslog / logrotate`
- `pkexec (Polkit)`
- `SSSD / LDAP / Kerberos`

✓ **Test it:** ausearch -k passwd_changes returns the recorded write event, confirming accounting is active.

Lab 14 · 3.1

AAA: sudo, PAM, and Polkit — Commands

Install sudo and create a user

```
apt update && apt install -y sudo
useradd -m -s /bin/bash alice
echo "alice:Passw0rd!" | chpasswd
id alice
```

Grant sudo via group and drop-in file

```
usermod -aG sudo alice
EDITOR=tee visudo -f /etc/sudoers.d/lab <<'EOF'
alice ALL=(ALL) NOPASSWD: /usr/bin/systemctl status *, /usr/bin/apt update
EOF
chmod 440 /etc/sudoers.d/lab
visudo -c
sudo -l -U alice
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 15 · Objective 3.2

Firewalls: ufw, firewalld, iptables, nftables, ipset

Maps to CompTIA Linux+ objective 3.2 — Configure and implement firewalls on a Linux system

Goal

Configure firewalls at every layer of the Linux netfilter stack, from the ufw wrapper down to iptables and nftables, and understand stateful inspection. Explore zones, rich rules, ipset, and SNAT/DNAT.

What you'll do

Configure ufw, firewalld zones, nftables, ipset, NAT rules, and a stateful conntrack pattern.

KEY COMMANDS & CONCEPTS

- `ufw allow/deny/enable`
- `firewalld zones and rich rules`
- `iptables / nftables chains`
- `ipset hash:ip`
- `SNAT / DNAT / MASQUERADE`
- `conntrack states (NEW/ESTABLISHED/RELATED)`

✓ **Test it:** ufw status and nft list ruleset show the loaded rules; conntrack states drive stateful acceptance.

Lab 15 · 3.2

Firewalls: ufw, firewalld, iptables, nftables, ipset — Commands

Install and configure ufw

```
apt update && apt install -y ufw
ufw allow 22/tcp
ufw deny 23/tcp
yes | ufw enable
ufw status verbose
```

Explore iptables backing ufw

```
iptables -L -n -v
iptables -t nat -L -n -v
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 16 · Objective 3.3

OS Hardening: Permissions, SELinux, SSH, Fail2ban

Maps to CompTIA Linux+ objective 3.3 — Apply operating system hardening techniques on a Linux system

Goal

Apply layered OS hardening from file permissions and immutability through SELinux labeling, then harden OpenSSH and add brute-force protection. Inventory SUID binaries and legacy cleartext services.

What you'll do

Set special permission bits, ACLs, and immutability, demo SELinux, harden SSH, and run fail2ban.

KEY COMMANDS & CONCEPTS

- `chmod setuid/setgid/sticky bit`
- `umask / chattr immutability`
- `POSIX ACLs (setfacl/getfacl)`
- `SELinux chcon/restorecon/audit2allow`
- `OpenSSH hardening (PermitRootLogin)`
- `fail2ban`
- `SUID hunting (find -perm -4000)`

✓ **Test it:** fail2ban-client status sshd shows the active jail and sshd -t confirms the hardened config is valid.

Lab 16 · 3.3

OS Hardening: Permissions, SELinux, SSH, Fail2ban — Commands

chmod, chown, special bits

```
useradd -m alice 2>/dev/null
mkdir -p /srv/lab && cd /srv/lab
cat >run.sh <<'EOF'
#!/bin/bash
id
EOF
chmod 750 run.sh
```

umask, chattr, lsattr

```
umask 027
touch /srv/lab/newfile && ls -l /srv/lab/newfile
chattr +i /etc/resolv.conf
lsattr /etc/resolv.conf
chattr -i /etc/resolv.conf
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 17 · Objective 3.4

Account Hardening

Maps to CompTIA Linux+ objective 3.4 — Explain account hardening techniques and best practices

Goal

Harden user accounts through password quality and aging policies, brute-force lockouts, and restricted or non-interactive shells. Explore TOTP MFA and Have I Been Pwned k-anonymity lookups.

What you'll do

Tune login.defs, enforce pwquality, set aging with chage, add pam_faillock, and lock down shells.

KEY COMMANDS & CONCEPTS

- `/etc/login.defs (PASS_MAX_DAYS)`
- `pam_pwquality / pwscore`
- `chage aging, passwd -l/-u`
- `pam_faillock lockout`
- `nologin / rbash restricted shells`
- `Google Authenticator TOTP MFA`
- `HIBP k-anonymity API`

✓ **Test it:** chage -l alice and passwd -S alice confirm aging and lock state; the HIBP query flags breached passwords.

Lab 17 · 3.4

Account Hardening — Commands

login.defs and a test user

```
apt update && apt install -y libpam-pwquality
useradd -m -s /bin/bash alice
echo "alice:Passw0rd!" | chpasswd
sed -i 's/^PASS_MAX_DAYS.*/PASS_MAX_DAYS   90/' /etc/login.defs
sed -i 's/^PASS_MIN_DAYS.*/PASS_MIN_DAYS   1/' /etc/login.defs
grep -E '^PASS_' /etc/login.defs
```

Password quality with pwquality

```
cat >/etc/security/pwquality.conf <<'EOF'
minlen = 14
dcredit = -1
ucredit = -1
lcredit = -1
ocredit = -1
retry = 3
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 18 · Objective 3.5

Cryptography: GPG, LUKS, OpenSSL, WireGuard

Maps to CompTIA Linux+ objective 3.5 — Explain cryptographic concepts and technologies in a Linux environment

Goal

Practice core Linux cryptography: GPG key generation and signing, LUKS2 disk encryption, OpenSSL certificates, hashing and HMAC, and WireGuard keys. Learn to identify weak TLS versions and ciphers.

What you'll do

Generate GPG keys, encrypt/sign, build a LUKS2 volume, create an OpenSSL cert, and make WireGuard keys.

KEY COMMANDS & CONCEPTS

- `GPG batch key generation, encrypt/sign`
- `LUKS2 (cryptsetup, loopback)`
- `OpenSSL genrsa / req -x509`
- `hashing vs HMAC`
- `WireGuard Curve25519 keys`
- `TLS version/cipher inspection (s_client)`

✓ **Test it:** gpg --verify confirms the signature and gpg -d recovers plaintext; the LUKS volume mounts only with the passphrase.

Lab 18 · 3.5

Cryptography: GPG, LUKS, OpenSSL, WireGuard — Commands

GPG key generation in batch mode

```
apt update && apt install -y gnupg
mkdir -p /root/.gnupg && chmod 700 /root/.gnupg
cat >/tmp/gpgparams <<'EOF'
%no-protection
Key-Type: RSA
Key-Length: 4096
Name-Real: Lab
```

Encrypt, decrypt, and detached-sign

```
echo "top secret payload" > /tmp/msg.txt
gpg --yes --trust-model always -r lab@example.com -e /tmp/msg.txt
gpg --yes -d /tmp/msg.txt.gpg
gpg --yes --detach-sign /tmp/msg.txt
gpg --verify /tmp/msg.txt.sig /tmp/msg.txt
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 19 · Objective 3.6

Compliance, Auditing, and File Integrity

Maps to CompTIA Linux+ objective 3.6 — Explain the importance of compliance and audit procedures

Goal

Establish file-integrity monitoring and run compliance and audit tooling to detect tampering, rootkits, and misconfigurations. Cover secure deletion, login banners, and OpenSCAP/CIS benchmarks.

What you'll do

Baseline files with AIDE, scan with rkhunter/lynis/nmap, verify packages, and set banners.

KEY COMMANDS & CONCEPTS

- `AIDE file-integrity baseline`
- `rkhunter rootkit scan`
- `debsums / rpm -V package verification`
- `lynis system audit`
- `nmap -sV service detection`
- `shred secure deletion`
- `OpenSCAP / CIS benchmarks / banners`

✓ **Test it:** aide --check flags the tampered /etc/hostname, proving the integrity baseline detects unauthorized change.

Lab 19 · 3.6

Compliance, Auditing, and File Integrity — Commands

AIDE baseline and tamper check

```
apt update && apt install -y aide
aideinit -y -f 2>/dev/null || aide --init
mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db
echo "tampered" >> /etc/hostname
aide --check | head -n 40
```

Rootkit hunting with rkhunter

```
apt install -y rkhunter
rkhunter --update || true
rkhunter --propupd -q
rkhunter --check --sk --rwo
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

DOMAIN 4

Automation, Orchestration, and Scripting

17% of the exam · Labs 20–24

4.1 Summarize automation and orchestration use cases and techniques

4.2 Given a scenario, perform automated tasks using shell scripting

4.3 Summarize Python basics used for Linux system administration

4.4 Given a scenario, implement version control using Git

4.5 Summarize best practices and responsible uses of AI

Lab 20 · Objective 4.1

Infrastructure as Code with Ansible

Maps to CompTIA Linux+ objective 4.1 — Summarize the use cases and techniques of automation and orchestration

Goal

Learn Infrastructure as Code using Ansible as the primary agentless configuration management tool, and recognize the wider IaC ecosystem (Puppet, OpenTofu, cloud-init, Kubernetes ConfigMaps). Build an idempotent playbook that deploys and configures nginx.

What you'll do

Install Ansible, write an inventory, run ad-hoc commands, and author an idempotent playbook with variables, templates, and handlers.

KEY COMMANDS & CONCEPTS

- `Ansible`
- `inventory`
- `playbook`
- `idempotency`
- `handlers/templates`
- `ansible-galaxy`
- `GitOps`
- `cloud-init`

✓ **Test it:** Idempotent playbooks converge state without unnecessary restarts; Ansible, Puppet, OpenTofu, cloud-init, and ConfigMaps each occupy a distinct IaC niche.

Lab 20 · 4.1

Infrastructure as Code with Ansible — Commands

Install Ansible and prepare workspace

```
apt update
apt install -y ansible
mkdir -p /root/lab && cd /root/lab
ansible --version
```

Create inventory pointing at localhost

```
cat > /root/lab/inventory.ini <<'EOF'
[web]
localhost ansible_connection=local
EOF

ansible -i /root/lab/inventory.ini all -m ping
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 21 · Objective 4.2

Bash Scripting

Maps to CompTIA Linux+ objective 4.2 — Perform automated tasks using shell scripting

Goal

Practice core Bash scripting constructs (variables, conditionals, loops, functions, option parsing, regex, file tests) and assemble a defensive backup script, then audit it with shellcheck.

What you'll do

Write a series of Bash scripts exercising each language feature, build a real /etc backup script, and lint everything with shellcheck.

KEY COMMANDS & CONCEPTS

- `shebang`
- `parameter expansion`
- `arrays`
- `test operators [[ ]]`
- `getopts`
- `IFS`
- `set -euo pipefail`
- `shellcheck`

✓ **Test it:** A robust script uses set -euo pipefail, quotes variables, and passes shellcheck cleanly.

Lab 21 · 4.2

Bash Scripting — Commands

Set up a scripts directory

```
mkdir -p /root/scripts && cd /root/scripts
apt update && apt install -y shellcheck
bash --version
```

Variables, command substitution, parameter expansion

```
cat > /root/scripts/basics.sh <<'EOF'
#!/usr/bin/env bash
name="Linux+"
today=$(date +%F)
greeting="${1:-hello}"
echo "$greeting $name on $today"
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 22 · Objective 4.3

Python for System Administration

Maps to CompTIA Linux+ objective 4.3 — Summarize Python basics used for Linux system administration

Goal

Use Python for common Linux admin tasks: create a virtual environment, exercise core data types, walk the filesystem, parse JSON, call a REST API with requests, and enforce PEP 8 with black and flake8.

What you'll do

Set up a venv, write scripts covering data types, os.walk, JSON parsing, and HTTP requests, then organize code into a package and lint it.

KEY COMMANDS & CONCEPTS

- `venv/pip`
- `core types (bool/int/float/str/list/dict)`
- `type hints`
- `os.walk`
- `json module`
- `requests`
- `black/flake8/PEP 8`

✓ **Test it:** A venv isolates dependencies, requests loudly fails on bad HTTP, and black+flake8 enforce PEP 8 automatically.

Lab 22 · 4.3

Python for System Administration — Commands

Check Python and create a virtual environment

```
python3 --version
apt update && apt install -y python3-venv python3-pip
python3 -m venv ~/lab-venv
source ~/lab-venv/bin/activate
which python
pip install --upgrade pip
```

Install dependencies

```
pip install requests black flake8
pip list
mkdir -p ~/pylab && cd ~/pylab
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 23 · Objective 4.4

Git Version Control

Maps to CompTIA Linux+ objective 4.4 — Implement version control using Git

Goal

Work through Git end to end: init, commit, branch, merge, resolve conflicts, stash, tag, rewrite history with reset and rebase, and push/pull against a local bare remote.

What you'll do

Create a working repo and a bare origin repo, then practice the full Git workflow from first commit through conflict resolution, rebasing, and remote push/pull.

KEY COMMANDS & CONCEPTS

- `git init/config`
- `branch/merge --no-ff`
- `conflict resolution`
- `stash/tag`
- `reset --soft/rebase`
- `bare remote`
- `push/pull --rebase`
- `.gitignore`

✓ **Test it:** Git tracks history through commits and branches; merge/rebase integrate work and pull --rebase keeps a linear remote history.

Lab 23 · 4.4

Git Version Control — Commands

Install Git and set identity

```
apt update && apt install -y git
git config --global user.name  "Linux Plus Learner"
git config --global user.email "learner@example.com"
git config --global init.defaultBranch main
git config --global --list
```

Initialise a repo and make the first commit

```
mkdir -p /root/lab && cd /root/lab
git init lab && cd lab
cat > README.md <<'EOF'
# Lab repo
EOF
cat > .gitignore <<'EOF'
*.log
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 24 · Objective 4.5

Responsible AI Use for Linux Administrators

Maps to CompTIA Linux+ objective 4.5 — Summarize best practices and responsible uses of artificial intelligence (AI)

Goal

Adopt a verify-before-paste workflow for AI-generated Linux commands: redact secrets before sending prompts to hosted models, lint and sandbox suggestions, weigh local vs hosted models, and draft a corporate AI usage policy.

What you'll do

Build a verification workspace, redact secrets in prompts, shellcheck and sandbox an AI-suggested command, and author a verification checklist plus an AI usage policy.

KEY COMMANDS & CONCEPTS

- `verify-before-paste`
- `secret redaction`
- `local vs hosted LLMs`
- `shellcheck`
- `API key via env var`
- `AI usage policy`
- `data governance`
- `attribution`

✓ **Test it:** Always verify before paste: redact secrets, shellcheck, sandbox, then run; choose local vs hosted models per policy.

Lab 24 · 4.5

Responsible AI Use for Linux Administrators — Commands

Set up a verification workspace

```
apt update && apt install -y shellcheck jq curl
mkdir -p /root/ai-lab && cd /root/ai-lab
shellcheck --version | head -1
```

Try a local LLM (optional)

```
curl -fsSL https://ollama.com/install.sh | sh || echo "Ollama install skipped"
(ollama serve >/tmp/ollama.log 2>&1 &) ; sleep 2
ollama pull tinyllama 2>/dev/null && \
  ollama run tinyllama "Write a one-line bash command to list the 5 largest files in /var/log" \
  || echo "Local model unavailable; continue with the placeholder API workflow."
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

DOMAIN 5

Troubleshooting

22% of the exam · Labs 25–30

5.1 Summarize monitoring concepts and configurations

5.2 Given a scenario, troubleshoot hardware, storage, and OS issues

5.3 Given a scenario, troubleshoot networking issues

5.4 Given a scenario, troubleshoot security issues

5.5 Given a scenario, troubleshoot performance issues

Lab 25 · Objective 5.1

Monitoring Concepts and Tools

Maps to CompTIA Linux+ objective 5.1 — Summarize monitoring concepts and configurations in a Linux system

Goal

Learn the monitoring vocabulary (SLA/SLO/SLI) and stand up a working monitoring stack to produce metrics, alerts, webhooks, health checks, and aggregated logs.

What you'll do

Install SNMP, node_exporter, Prometheus and Alertmanager, then write and validate alert rules, test webhooks, and inspect journald logs.

KEY COMMANDS & CONCEPTS

- `SLA vs SLO vs SLI`
- `SNMP OID/MIB walk`
- `Prometheus scrape config`
- `node_exporter metrics endpoint`
- `promtool check rules`
- `webhook alerting`
- `curl health checks`
- `journalctl log aggregation`

✓ **Test it:** node_exporter, Prometheus, and its health/ready endpoints all respond, and promtool confirms the alert rule is valid.

Lab 25 · 5.1

Monitoring Concepts and Tools — Commands

Install SNMP and walk the system MIB

```
apt update
apt install -y snmpd snmp snmp-mibs-downloader
systemctl enable --now snmpd
snmpwalk -v 2c -c public localhost 1.3.6.1.2.1.1
```

Install and probe node_exporter

```
apt install -y prometheus-node-exporter
systemctl enable --now prometheus-node-exporter
ss -tlnp | grep 9100
curl -s http://localhost:9100/metrics | head -n 20
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 26 · Objective 5.2

Troubleshooting Hardware, Storage, and OS Issues

Maps to CompTIA Linux+ objective 5.2 — Analyze and troubleshoot hardware, storage, and Linux OS issues

Goal

Deliberately inject common OS and storage failures then diagnose and repair each, keeping every breakage contained to loopback devices, throwaway units, or subshells.

What you'll do

Reproduce a full filesystem, inode exhaustion, GRUB validation, a runaway CPU process, a failed systemd unit, a broken PATH, and a memory leak, then fix them.

KEY COMMANDS & CONCEPTS

- `ENOSPC / df -h`
- `inode exhaustion / df -i`
- `grub-mkconfig validation`
- `runaway process / top -bn1`
- `failed systemd unit`
- `broken PATH recovery`
- `memory leak / ps rss / free`

✓ **Test it:** Each fault is confirmed by the right counter (df, df -i, top, systemctl status, ps rss) and reversed cleanly without touching the host.

Lab 26 · 5.2

Troubleshooting Hardware, Storage, and OS Issues — Commands

Break: fill a filesystem (ENOSPC)

```
mkdir -p /mnt/tinyfs
dd if=/dev/zero of=/tmp/tiny.img bs=1M count=64
mkfs.ext4 -F /tmp/tiny.img
mount -o loop /tmp/tiny.img /mnt/tinyfs
dd if=/dev/zero of=/mnt/tinyfs/fill bs=1M count=200 || echo "ENOSPC as expected"
df -h /mnt/tinyfs
du -shx /mnt/tinyfs/* | sort -h
```

Break: inode exhaustion

```
df -i /mnt/tinyfs
mkdir /mnt/tinyfs/many
(cd /mnt/tinyfs/many && for i in $(seq 1 20000); do : > f$i 2>/dev/null || break; done)
df -i /mnt/tinyfs
rm -rf /mnt/tinyfs/many
df -i /mnt/tinyfs
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 27 · Objective 5.3

Troubleshooting Network Connectivity

Maps to CompTIA Linux+ objective 5.3 — Analyze and troubleshoot networking issues on a Linux system

Goal

Break and repair the common network faults (DNS, routing, firewall, MTU, IP, link, dual-stack) while practicing the standard L3/L4 triage toolkit.

What you'll do

Baseline the host, then inject DNS, gateway, firewall, MTU, duplicate-IP, and link-down faults, and finish with IPv4/IPv6 and packet-capture triage.

KEY COMMANDS & CONCEPTS

- `ip addr / ip route baseline`
- `resolv.conf / dig`
- `default gateway`
- `ufw port block`
- `MTU / ping -M do`
- `duplicate IP / arping`
- `dummy interface link toggle`
- `tcpdump / mtr / traceroute / nmap`

✓ **Test it:** Each single-variable fault produces its expected symptom and reverts cleanly, and IPv4/IPv6 plus capture tools confirm reachability.

Lab 27 · 5.3

Troubleshooting Network Connectivity — Commands

Baseline and install tools

```
apt update
apt install -y dnsutils iproute2 iputils-ping mtr-tiny traceroute nmap tcpdump ufw netcat-openbsd
ip -br addr
ip route
cat /etc/resolv.conf
ss -tulpn | head
```

Break: DNS, then fix

```
cp /etc/resolv.conf /tmp/resolv.conf.bak
sed -i 's/^nameserver/#nameserver/' /etc/resolv.conf
dig +time=2 +tries=1 example.com || echo "DNS broken as expected"
echo 'nameserver 8.8.8.8' > /etc/resolv.conf
dig +short example.com
cp /tmp/resolv.conf.bak /etc/resolv.conf
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 28 · Objective 5.4

Troubleshooting Security Issues

Maps to CompTIA Linux+ objective 5.4 — Analyze and troubleshoot security issues on a Linux system

Goal

Diagnose and repair security faults spanning SELinux contexts, POSIX ACLs, certificate trust, account lockout, broken repos, and weak TLS, using a Rocky container for the SELinux portion.

What you'll do

Pull a Rocky container for SELinux practice, then break and fix ACLs, CA trust, pam_faillock lockout, a bad apt repo, and weak TLS on the Ubuntu host.

KEY COMMANDS & CONCEPTS

- `SELinux context / chcon / restorecon`
- `AVC denials / ausearch / audit2allow`
- `POSIX ACL / getfacl / setfacl`
- `CA trust / update-ca-certificates`
- `pam_faillock lockout`
- `apt repo / TLS s_client`
- `unattended-upgrades`

✓ **Test it:** restorecon fixes SELinux type, getfacl/setfacl explains hidden ACL denials, the CA trust bundle rebuilds, faillock resets, and modern TLS negotiates while TLS1.0 is refused.

Lab 28 · 5.4

Troubleshooting Security Issues — Commands

Prep host and pull a Rocky container

```
apt update
apt install -y docker.io acl openssl libpam-modules ca-certificates
systemctl start docker
docker pull rockylinux:9
docker run -dit --name selab --privileged rockylinux:9 bash
docker exec selab dnf -y install policycoreutils policycoreutils-python-utils selinux-policy-targeted setools-console audit
httpd
```

Break: SELinux context, then restorecon

```
docker exec selab bash -c '
mkdir -p /var/www/html && echo hello > /tmp/index.html
cp /tmp/index.html /var/www/html/index.html
ls -Z /var/www/html/index.html
chcon -t user_home_t /var/www/html/index.html
ls -Z /var/www/html/index.html
restorecon -v /var/www/html/index.html
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 29 · Objective 5.5

Troubleshooting Performance Issues

Maps to CompTIA Linux+ objective 5.5 — Analyze and troubleshoot performance issues

Goal

Stress CPU, memory, disk I/O, and network in turn, then walk the universal performance loop: baseline, spike, identify the bottleneck with the right counter, and resolve it.

What you'll do

Install stress-ng, fio, and sysstat, baseline the host, then generate controlled CPU, memory, disk, and network spikes and attribute each to the right counter and PID.

KEY COMMANDS & CONCEPTS

- `baseline: uptime/free/vmstat/mpstat`
- `stress-ng CPU spike / load average`
- `memory pressure / vmstat si-so`
- `fio disk storm / iostat -x`
- `D-state / ping jitter`
- `pidstat context switches`
- `ps --sort triage script`

✓ **Test it:** Each spike maps to its diagnostic counter (load vs mpstat, si/so vs free, %util vs await) and the triage script shows baseline-to-recovery.

Lab 29 · 5.5

Troubleshooting Performance Issues — Commands

Install the toolkit and baseline

```
apt update
apt install -y stress-ng fio sysstat htop iotop iftop dstat procps
uptime
free -m
vmstat 1 3
mpstat 1 2
df -h /
```

Generate and observe a CPU spike

```
stress-ng --cpu 4 --timeout 30 &
sleep 5
uptime
mpstat -P ALL 1 3
top -bn1 | head -n 15
wait
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Lab 30 · Objective Capstone

Capstone: End-to-End Server Build and Triage

Maps to CompTIA Linux+ objective Capstone — Capstone: End-to-end server build & triage (all domains)

Goal

Integrate Objectives 1-5 into one coherent build: a hardened admin account, LVM storage, nginx with a systemd override, a locked firewall, a health-check timer, and git-versioned config, then inject a fault and triage it end-to-end.

What you'll do

Provision webops with hardened SSH, build LVM-backed /srv/web, deploy nginx, lock ufw to web ports, add a health-check timer, version /etc/nginx in git, then inject and remediate a disk-full fault.

KEY COMMANDS & CONCEPTS

- `hardened SSH / narrow sudo`
- `LVM: pvcreate/vgcreate/lvcreate`
- `systemd drop-in override`
- `ufw default-deny allowlist`
- `systemd timer health check`
- `git-versioned /etc/nginx`
- `fault injection and triage loop`

✓ **Test it:** After remediation curl returns the Capstone page with a 200, confirming the full build and the status/journal/sockets/storage/probe triage loop.

Capstone: End-to-End Server Build and Triage — Commands

Provision a user and harden SSH

```
apt update
apt install -y openssh-server sudo nginx lvm2 ufw git curl
useradd -m -s /bin/bash -G sudo webops
echo 'webops:LabPass!23' | chpasswd
echo 'webops ALL=(ALL) NOPASSWD: /bin/systemctl, /usr/bin/journalctl' >/etc/sudoers.d/webops
sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
```

Build LVM-backed /srv/web over loopback

```
dd if=/dev/zero of=/var/lab-pv.img bs=1M count=200
LOOP=$(losetup --find --show /var/lab-pv.img)
pvcreate $LOOP
vgcreate labvg $LOOP
lvcreate -n weblv -L 150M labvg
mkfs.ext4 /dev/labvg/weblv
mkdir -p /srv/web && mount /dev/labvg/weblv /srv/web
```

Full step-by-step for all steps is in the Learner Guide and the labs/ folder.

Summary & Q&A

- You have covered all five CompTIA Linux+ XK0-006 domains through 30 hands-on labs.
- System Management · Services & User Management · Security · Automation & Scripting · Troubleshooting.
- Keep practising on Killercoda and review the Learner Guide before the exam.
- Questions?

Courseware & Assessment on the LMS

- Download the slide deck and Learner Guide from the LMS.
- Complete the Written Assessment and Practical Performance on the LMS.
- LMS: https://lms-tms.tertiaryinfotech.com/
- Contact: enquiry@tertiaryinfotech.com · +65 6100 0613 · www.tertiarycourses.com.sg

Certification & TRAQOM Survey

- Scan the TRAQOM QR code on the LMS and complete the course feedback survey.
- A WSQ Statement of Attainment / certificate follows a Competent assessment result.
- Your feedback helps us improve — thank you!
- Survey & certification portal: ai-lms-tms.tertiaryinfo.tech

Thank You!

All the best for your CompTIA Linux+ XK0-006 certification.